Topic – Inviare e ricevere messaggi tra nodi

Fondamenti di Robotica

Docenti: Loredana Zollo
Clemente Lauretti
Tutor: **Rita Molle**
email: rita.molle@unicampus.it



UNIVERSITA'
CAMPUS
BIO-MEDICO
DI ROMA



CAMPUS BIO-MEDICO UNIVERSITY OF ROME
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

Outline

- Recap topic
- Esercitazione
- Scrivere un publisher
- Scrivere un subscriber

Recap topic

- Un topic permette la comunicazione tra nodi tramite publisher/subscriber
- Un topic è definito da **name** e **interface**
- Il name deve iniziare con una lettera
- **Publisher:** nodi che inviano il dato sul topic
- **Subscriber:** nodi che leggono dal topic
- publisher e subscriber di un topic devono avere la stessa **interface**
- publisher e subscriber sono anonimi
- Un nodo può contenere più publisher e subscriber a differenti topic

- Per creare un publisher e un subscriber bisogna importare l'interface nel codice

- Per pubblicare bisogna creare un messaggio per il publisher

- Per il subscriber bisogna creare una funzione di callback → si ricevono e processano i messaggi in entrata

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Esercitazione:

- Scrivere un nodo che abbia sia publisher che subscriber
  - Si sottoscrive ai topic della camera
  - Pubblica le pose degli ArUco sia in terna base che in terna camera
- Visualizzazione delle pose su RViz

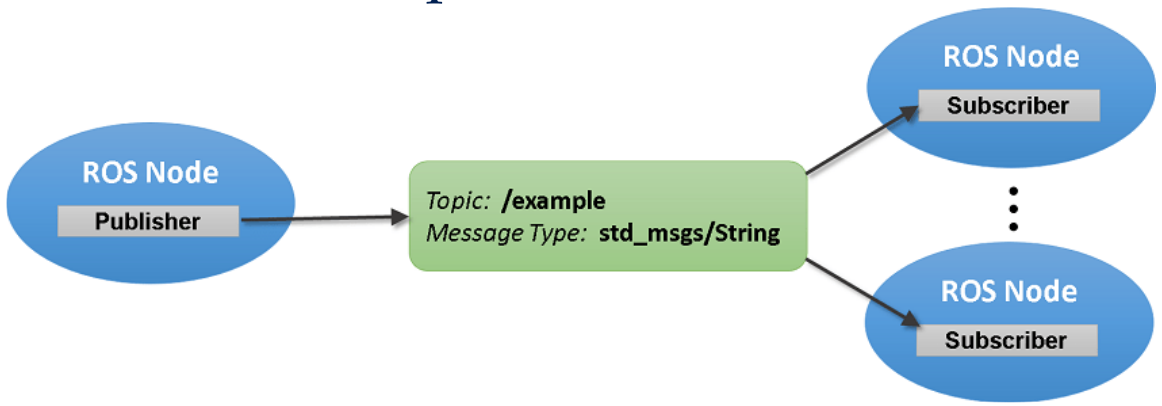
Scrivere un publisher – Python

Andare nel pkg, creare un file Python e renderlo eseguibile

Terminal 1:

```
cd ~/progetto_n_ws/src/aruco_pkg/aruco_pkg/
touch aruco_detection.py
chmod +x aruco_detection.py
```

Aprire il file .py e scrivere il nodo publisher



Importare il tipo di messaggio → interface

```
from geometry_msg.msg import PoseStamped
```

Nome pacchetto — Nome della cartella — Classe dell'interface

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Scrivere un publisher – Python

Usare la funzione

create_publisher(*topic_interface*, "*topic_name*", queue size)

Fare il build del pkg in cui è il nodo

Terminal 1: cd ~/progetto_*n*_ws/
colcon build

Aggiungere il source al /.bashrc

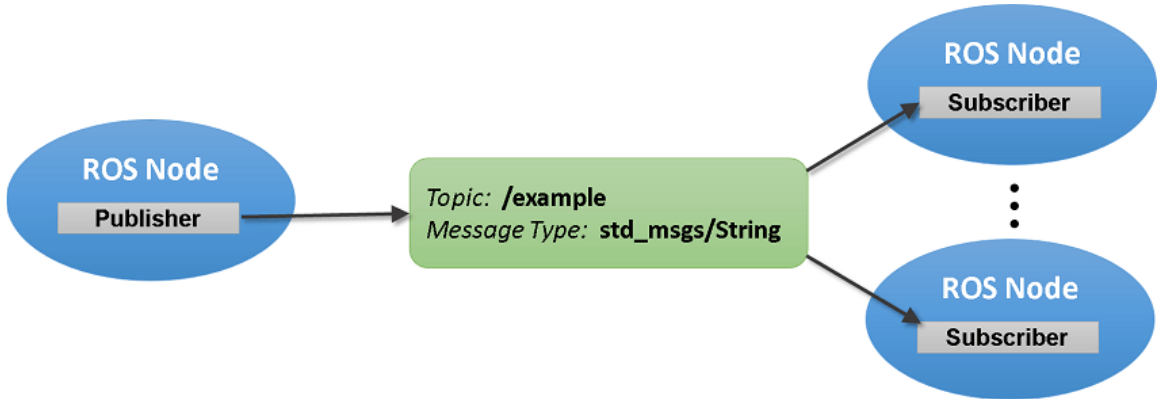
Runnare il nodo

Terminal 1: source ~/.bashrc
ros2 run aruco_pkg aruco_detection

Terminal 2: ros2 topic list
ros2 topic echo /<*topic_name*> → vedere il dato pubblicato
ros2 topic echo /aruco_camera_1

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Scrivere un subscriber – Python



Aprire il file .py e scrivere la parte del subscriber

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Scrivere un subscriber – Python

Usare la funzione
create_subscription(*topic_interface*, "*topic_name*", self.*callback_topic*, queue size)

Creare una callback per sottoscrivere il topic → il topic è ricevuto nella callback, e può essere qui processato e usato

Nella callback del subscriber →il messaggio è ricevuto nei parametri della funzione

```
def callback_topic(self, msg: topic_interface):
        # CODICE #
```


E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ